Weekly Meeting Update

Repository Structure Update

I set up our GitHub organization with three repositories (`job-orchestrator`, `batch-workload`, and `platform-infra`) instead of splitting every architectural component into its own repo. My reasoning behind this decision is that 1:1 split (one repo per part of the system) would possibly have lead to dependency version churn, complicated local testing significantly, and slowed down our Phase 1 implementation speed. Grouping by operational lifecycle keeps our control-plane services together, isolates the batch pod worker, and separates our environment configurations cleanly.

Repositories Added & Their Basic Structure

- **job-orchestrator**
  - `common/` Shared schemas (JobMessage), interface contracts, and exit codes
  - `producer/` Scheduled CSV file generator and queue publisher
  - `consumer/` Redis Stream consumer and concurrency gate
  - `scheduler/` Kubernetes Job spec builder and submitter
  - `status-tracker/` Pod event watcher and MySQL state reconciler
  - `api/` REST API for job submission, status querying, and retries
- **batch-workload**
  - Worker pod application (streams CSV from storage, validates, executes batched JDBC writes to MySQL, and checkpoints progress)
  - Multi-stage Dockerfile (builds the runtime container image)
- **platform-infra**
  - `compose/` Local Docker Compose configuration for MySQL 8, Redis (AOF), and MinIO
  - `migrations/` Flyway SQL migration scripts (`V1__initial_schema.sql`)
  - `scripts/` Local development startup and teardown scripts
  - Deployment manifests, Helm charts, and Prometheus/Grafana dashboard configurations

Sprint Setup Tasks on the Board

I added six initial setup and foundational tasks to the board. Rather than building all of this out myself, I wanted to open them up so everyone can get hands-on with the repositories early. Some of you have solid experience with Docker and Kubernetes, so taking on INFRA-1 and INFRA-2 would be a great fit. I've already assigned myself to CI-1 and CORE-1, and I'm

currently mapping out the next tickets for the synthetic data generator and the producer pipeline in `job-orchestrator`.

Tasks Added as of Last Week

- **INFRA-1:** Local Backing Services Compose Setup (`platform-infra`)
    - Provides a Docker Compose environment running MySQL 8, Redis with AOF persistence, and MinIO locally. It includes healthchecks and startup scripts so anyone can boot the required backing services with one command.
- **INFRA-2:** MySQL V1 DDL Migrations (`platform-infra`)
    - Establishes the baseline relational schema defining the job, `job_attempt`, `pod_event`, `file_progress`, and `account_payment` tables. It also includes the composite unique constraint required to guarantee idempotent record loading during retries.
- **CORE-1:** Multi-Module Java 17 Project Scaffold (`job-orchestrator`)
    - Configures the root build setup with submodules for common, producer, consumer, status-tracker, and api. This ensures the entire control plane compiles cleanly together while maintaining modular code boundaries.
- **CORE-2:** Implement /common Schemas, Exit Codes & Contracts (`job-orchestrator`)
    - Defines shared DTOs like the immutable JobMessage record along with standard workload exit codes. It also establishes the core Java interface contracts for queue, storage, and repository interactions.
- **WORK-1:** Workload Project Scaffold & Multi-Stage Dockerfile (`batch-workload`)
    - Initializes the standalone Java application that will run inside the Kubernetes pod to process batch files. It sets up basic environment variable ingestion and includes a multi-stage Dockerfile to generate a minimal container image.
- **CI-1:** GitHub Actions PR Verification Workflows (`job-orchestrator`)
    - Adds automated CI checks that trigger on pull requests against main. It verifies code formatting, compiles the Java 17 projects, and runs the test suite to ensure broken code is never merged.